BillFlow Platform

Enterprise Invoicing & Business Billing Management System

A production-ready, full-stack micro-enterprise billing platform featuring automated multi-tier GST calculations, comprehensive customer ledger accounting, real-time financial reporting, client self-service portal, multi-container Docker architecture, and high-availability Kubernetes cluster orchestration.

KEY SYSTEM HIGHLIGHTS

- Full-Stack Decoupled Architecture: React 18 Vite SPA + Node.js 20 Express REST API
- 15 Built Pages: CRM, Invoicing, Ledger, Payments, Credit Notes, Reports, Portal, Users
- Multi-Container Docker Orchestration: Node.js 20 Alpine, Nginx SPA Reverse Proxy, Mongo 6
- Kubernetes (K8s) Cluster: Pods, Persistent Volumes (PVC), NodePort/ClusterIP, Ingress

PROJECT METADATA & REPOSITORY INFORMATION

GitHub Remote URL: https://github.com/prakashkommarajula-dot/Billing-Platform.git
Repository Branch: main (Latest Pushed & Verified)
Local Execution: Frontend :80 / :5173 / :3000 | Backend :5000 | MongoDB :27018
Kubernetes Cluster: Frontend NodePort :30080 | Backend NodePort :30500 | Namespace: billflow
Date & Version: November 2026 | Release v1.0.0 Production

1. Project Overview & Architecture

Objectives, System Topology & Component Interactions

1.1 Executive Summary & Core Objectives

BillFlow is an enterprise-grade web application engineered to solve critical operational bottlenecks encountered by growing businesses: manual tax calculation errors, unorganized client billing histories, delayed receivable collections, and lack of transparency. The platform bridges the gap between client relationship management (CRM) and financial accounting by providing real-time ledger updates, automated GST computation, and seamless invoice-to-settlement workflows.

- **Automated Multi-Slab GST Engine:** Dynamically computes CGST + SGST (intra-state) or IGST (inter-state) based on customer state origin and product HSN/SAC codes across 0%, 5%, 12%, 18%, and 28% slabs.
- **Integrated Client Ledger Accounting:** Maintains an immutable double-entry style chronological ledger detailing Opening Balance, Invoices (+), Payments (-), and Credit Notes (-) with accurate running balances.
- **Secure Role-Based Access Control:** Enforces strict API and UI boundary isolation between Business Owners, Staff Accountants, Platform Admins, and Customer Portal users.
- **Instant PDFKit Document Generation:** Generates server-side vector PDF tax invoices, credit notes, and account statements directly in the browser with 1-click download capabilities.

1.2 High-Level Multi-Tier Architecture

The application follows a modern cloud-native decoupled 3-tier architecture designed for horizontal scalability, zero-downtime rolling updates, and complete failure isolation.

Tier / Layer	Component Name	Underlying Technology	Network & Port Mapping
Presentation Tier	billflow-frontend	React 18, Vite 6, Nginx 1.31	Host Port 80, 5173, 3000 (K8s NodePort 30080)
Gateway / Proxy	Nginx Reverse Proxy	Nginx Alpine (try_files)	Internal proxy pass of /api/* -> backend:5000
Application Tier	billflow-backend	Node.js 20, Express 4, Mongoose 8	Port 5000 (K8s NodePort 30500)
Persistence Tier	billflow-mongodb	MongoDB 6 Jammy + PVC Volume	Port 27017 (K8s ClusterIP / Host 27018)
Security Tier	JWT & Bcrypt Engine	HMAC SHA-256 + Salted Hashing	Stateless Bearer Header in all REST calls

2. Technology Stack & Directory Structure

Comprehensive Breakdown of Tools, Libraries & Codebase Organization

2.1 Complete Technology Matrix

Frontend Technologies

Package / Library	Version	Role & Architectural Responsibility
React	^18.3.1	Component-driven reactive UI library with Hooks state management
Vite	^6.0.5	Next-generation ES module bundler and fast HMR development server
React Router DOM	^6.28.0	Declarative client-side routing with role-guarded private routes
Lucide React	^0.468.0	Scalable modern vector icon suite across all 15 page interfaces
Recharts	^2.15.4	Interactive SVG charts for monthly revenue trends and receivables aging
CSS3 Custom Variables	Native	Zero-dependency design system with responsive layouts and tokens

Backend & Database Technologies

Package / Library	Version	Role & Architectural Responsibility
Node.js LTS	20.x Alpine	High-performance asynchronous JavaScript server runtime
Express	^4.21.2	Lightweight REST API framework for routing and middleware pipelines
Mongoose	^8.9.5	ODM library for MongoDB schema enforcement, hooks, and indexing
JSON Web Tokens (JWT)	^9.0.2	Stateless HMAC-SHA256 authenticated session token generation
bcryptjs	^3.0.2	Cryptographic salted hashing for user credentials and passwords
PDFKit	^0.20.2	High-fidelity server-side vector PDF generator for invoices & receipts
MongoDB	6.0 Jammy	Document-oriented database with ACID transactions and schema validation

2.2 Clean Repository Separation

The project maintains strict structural separation between frontend and backend source trees to prevent dependency leakage and support independent CI/CD pipelines.

- **backend/:** Contains Express routes, controllers, Mongoose schemas, auth middleware, and PDF generator.
- **frontend/:** Contains React pages, layout wrappers (Sidebar, Navbar), services/api.js, and CSS files.
- **k8s/:** Houses all declarative Kubernetes manifests including Deployments, Services, ConfigMaps, and Ingress.
- **docker-compose.yml:** Root orchestration file connecting MongoDB, Backend, and Frontend containers.

3. Detailed Module Specifications (Part 1)

Implementation Details for Pages 1 through 7

Page 1 – Secure Authentication & 1-Click Role Switcher

• Route: /login | Backend: POST /api/auth/login

Features: JWT Bearer authentication with encrypted password comparison. Includes 1-click preset login buttons for instant evaluation across Business Owner, Accountant, Admin, and Customer roles.

Page 2 – Multi-Step Business Registration

• Route: /register | Backend: POST /api/auth/register

Features: Comprehensive business onboarding form collecting Full Name, Legal Company Name, Email, Phone, Business Address, State, Postal Code, and GSTIN. Enforces strict input validation and role assignments.

Page 3 – Executive Financial Dashboard

• Route: /dashboard | Backend: GET /api/dashboard/stats

Features: 4 KPI summary cards (Total Revenue, Outstanding Receivables, Total Invoices, Overdue Accounts), interactive Recharts Monthly Revenue Trend chart, Recent Invoices table, and quick creation action pills.

Page 4 – Customer Relationship Management (CRM)

• Route: /customers | Backend: GET /api/customers, POST /api/customers

Features: Client directory displaying Customer ID, Company Name, Contact Person, Email, Phone, City, GSTIN, and Outstanding Balance. Filterable by status (Active/Inactive), search by name or GSTIN.

Page 5 – Customer Profile & Financial Ledger

• Route: /customers/:id or /customers/details | Backend: GET /api/customers/:id

Features: 360-degree client view displaying billing terms and an audited chronological financial ledger table detailing Opening Balance, Invoices (+), Payments (-), and Credit Notes (-) with real-time running balances.

Page 6 – Products & Services Catalog

• Route: /products | Backend: GET /api/products, POST /api/products

Features: Item catalog supporting SKU codes, Category (Goods vs Services), Unit Price, HSN/SAC classification, and GST rate assignment (0%, 5%, 12%, 18%, 28%) with real-time stock and pricing edits.

Page 7 – Dynamic Tax Invoice Generator

• Route: /invoices/create | Backend: POST /api/invoices

Features: Multi-line invoice creator with automated numbering (INV-YYYY-XXXX), customer selection, item lookups, automated CGST/SGST vs IGST calculation based on customer state, discounts, and payment terms.

4. Detailed Module Specifications (Part 2)

Implementation Details for Pages 8 through 15

Page 8 – Invoice Details & Vector PDF Export

• Route: /invoices/:id | Backend: GET /api/invoices/:id, GET /api/invoices/:id/pdf

Features: Formal A4 invoice preview showing invoice status (Draft, Sent, Paid, Partially Paid, Overdue), itemized tax breakdown, payment settlement history, and instant browser download of a vector PDF.

Page 9 – Payment Collections & Receipt Slips

• Route: /payments | Backend: GET /api/payments, POST /api/payments

Features: Record full or partial settlements against outstanding invoices with transaction references, payment method (Bank Transfer, UPI, Credit Card, Cheque, Cash), and immediate balance recalculation.

Page 10 – Credit Notes Management

• Route: /credit-notes | Backend: GET /api/credit-notes, POST /api/credit-notes

Features: Issue credit adjustments linked to original invoices. Includes validation to prevent credit amounts exceeding outstanding balance, ledger updates, and downloadable credit note PDF generation.

Page 11 – Reports & Analytics (4 Reporting Tabs)

• Route: /reports | Backend: GET /api/reports/*

Features: 1) Sales Invoice Report, 2) Outstanding Receivables Aging Report, 3) Tax Summary Report (CGST, SGST, IGST totals), and 4) Customer Transaction Ledger. Supports date filtering, CSV export, and PDF printing.

Page 12 – Dedicated Client Self-Service Portal

• Route: /portal | Backend: GET /api/portal/*

Features: Client-scoped dashboard allowing customers to view their invoices, check outstanding balances, inspect payment history, and download statements without administrative access.

Page 13 – User Management & Role-Based Access Control

• Route: /users | Backend: GET /api/users, POST /api/users

Features: Administrative user table, role assignment (Admin, Business Owner, Accountant, Customer), account status toggling (Active vs Suspended), and secure password reset workflows.

Page 14 – Business Settings & Legal Compliance

• Route: /settings | Backend: GET /api/settings, PUT /api/settings

Features: Configure enterprise business branding, logo upload, registered GSTIN, invoice prefix, next numbering sequence, default tax slabs, and custom terms & conditions.

Page 15 – User Profile & Account Security

• Route: /profile | Backend: GET /api/profile, PUT /api/profile

Features: Manage personal profile information, update profile picture, and change passwords with current password verification and bcrypt hashing.

5. Docker & Containerization

Multi-Stage Builds, Nginx Reverse Proxy & Compose Orchestration

5.1 Multi-Container Docker Architecture

BillFlow is containerized into a cohesive three-service architecture managed via Docker Compose. The setup guarantees identical runtime environments across local development and production deployments.

Container Service	Base Docker Image	Build Strategy & Config	Port & Volume Bindings
billflow-mongodb	mongo:6-jammy	Official Jammy image with automated ping healthcheck	Host: 27018:27017 | Vol: billflow_mongodb_data
billflow-backend	node:20-alpine	Alpine base, native build tools, production deps only	Host: 5000:5000 | Env: MONGO_URI, JWT_SECRET
billflow-frontend	nginx:alpine	Multi-stage build (Stage 1 Vite -> Stage 2 Nginx)	Host: 80, 5173, 3000 -> Container Port 80

5.2 Frontend Multi-Stage Build & Reverse Proxy

The frontend Dockerfile utilizes a multi-stage compilation pattern. Stage 1 executes `npm run build` using Node 20 to generate optimized static chunks in `/app/dist`. Stage 2 copies these compiled assets into a minimal Nginx Alpine image, resulting in a compressed 26MB production image. Nginx is configured to handle SPA routing fallback (`try_files $uri /index.html`) and reverse proxy all `/api/*` HTTP requests directly to `http://backend:5000/api/`.

5.3 Essential Docker CLI Commands

Run the following commands in the workspace root:

- **docker compose build:** Compiles frontend Vite bundle and installs backend production dependencies.
- **docker compose up -d:** Starts MongoDB, Backend API, and Frontend Nginx in detached mode.
- **docker compose ps:** Verifies running containers, port bindings, and healthy status.
- **docker compose logs -f:** Streams live container execution logs across all services.
- **docker compose down:** Gracefully stops and terminates container instances.

6. Kubernetes (K8s) Orchestration

Declarative Manifests, Pods, Services, PVCs & Ingress

6.1 Kubernetes Workload Topology

The application is fully container-orchestrated using Kubernetes manifests located in the `k8s/` directory. All resources are encapsulated within a dedicated `billflow` namespace.

K8s Resource Type	Resource Name	Replicas / Spec	Purpose & Target Endpoints
Namespace	billflow	1	Logical isolation boundary for all application workloads
ConfigMap	billflow-config	N/A	Centralized config: PORT, NODE_ENV, MONGODB_URI
Secret	billflow-secret	N/A	Opaque cryptographically secured JWT_SECRET
PVC	mongodb-pvc	1Gi (RWO)	Standard storage claim for database persistence
Deployment	mongodb	1 Replica	MongoDB 6 with TCP socket readiness/liveness probes
Service	mongodb	ClusterIP	Internal cluster DNS discovery on port 27017
Deployment	backend	2 Replicas	Express REST API with HTTP /api/health probes
Service	backend	NodePort (30500)	External NodePort access & internal ClusterIP
Deployment	frontend	2 Replicas	Nginx SPA containers with HTTP / health probes
Service	frontend	NodePort (30080)	External NodePort access for web browser traffic
Ingress	billflow-ingress	N/A	Routing rules forwarding / and /api/ to services

6.2 Kubernetes Deployment Commands

- **kubectl apply -k k8s/:** Deploys all resources via Kustomize in a single declarative command.
- **kubectl get all,pvc -n billflow:** Displays status of all Pods, Services, Deployments, and PVCs.
- **kubectl rollout status deployment backend -n billflow:** Monitors zero-downtime rolling updates.
- **kubectl logs -l app=backend -n billflow -f:** Streams logs across all active backend replica pods.
- **kubectl delete -k k8s/:** Gracefully terminates and cleans up cluster workloads.

7. Verification, Testing & Quick Reference

End-to-End Validation, Port Matrix & Evaluation Presets

7.1 Live Execution & Testing Results

All application layers have undergone automated and end-to-end integration testing:

- **Health Check Endpoints:** Backend HTTP probe at /api/health returns HTTP 200 with service uptime.
- **Authentication & Registration:** Verified user registration, bcrypt password hashing, and JWT token issuance.
- **Database Operations:** Verified Customer creation (CUST-001) and MongoDB persistence via PVC.
- **Reverse Proxy Routing:** Verified Nginx reverse proxy routes all /api/ endpoints to backend container.
- **Kubernetes NodePort Access:** Verified browser access at http://localhost:30080 and API at :30500.

7.2 Pre-Configured Demo Credentials

Role	Email Address	Password	Default Landing Route
Business Owner	owner@apexenterprises.com	Password123!	/dashboard (Executive KPIs & Charts)
Customer Portal	customer@globaltech.com	Password123!	/portal (Client Invoices & Statement)
System Admin	admin@platform.com	Password123!	/users (User Management & Roles)
Accountant	accountant@apexenterprises.com	Password123!	/reports (Financial & Tax Ledgers)

7.3 Port Matrix Quick Reference

Service / Component	Local Dev Port	Docker Compose Port	Kubernetes NodePort
Frontend Web App	http://localhost:5173	http://localhost:80 (5173/3000)	http://localhost:30080
Backend REST API	http://localhost:5000	http://localhost:5000	http://localhost:30500
MongoDB Database	localhost:27017	localhost:27018	Internal ClusterIP :27017